

Proyecto Identificación y Pixelado de Rostros en Imágenes con Arquitectura Event-Driven

1. Definición del proyecto

El proyecto consistirá en el desarrollo de un sistema distribuido basado en eventos (Event-Driven Architecture) que procese imágenes para:

- Detectar rostros en imágenes
- Estimar la edad de cada rostro
- Pixelar automáticamente los rostros de personas menores de 18 años

A diferencia del enfoque tradicional basado en llamadas HTTP síncronas, este sistema utilizará un bus de eventos Kafka para la comunicación entre servicios.

2. Requisitos Técnicos

Arquitectura basada en eventos

Todos los servicios deben comunicarse mediante un sistema de mensajería:

- Kafka
- Comunicación asíncrona mediante topics
- Patrón publish/subscribe

Contenedores Docker

1. API Gateway (Productor de eventos)

- **Función:**
 - Recibir imágenes (HTTP REST)
 - Publicarlas en un topic (ej: `images.raw`)
- **Tecnología:** FastAPI o Flask

2. Orquestador (Consumidor + Productor)

- **Función:**
 - Coordinar el flujo mediante eventos
 - Enviar imágenes a los siguientes topics según estado
- **Alternativa:** flujo completamente desacoplado sin orquestador

3. Face Detection Service (Consumidor + Productor)

- **Consume:** `images.raw`

- **Produce:** `images.faces_detected`
- **Función:**
 - Detectar rostros
 - Generar bounding boxes
- **Tecnología:** OpenCV, Dlib, YOLO

4. Age Detection Service (Consumidor + Productor)

- **Consume:** `images.faces_detected`
- **Produce:** `images.age_estimated`
- **Función:**
 - Estimar edad de cada rostro
 - Clasificar: `<18` o `>=18`
- **Tecnología:** TensorFlow / PyTorch (ResNet o similar)

5. Pixelation Service (Consumidor + Productor)

- **Consume:** `images.age_estimated`
- **Produce:** `images.processed`
- **Función:**
 - Pixelar rostros de menores
- **Tecnología:** OpenCV

6. Storage / Result Service (Consumidor)

- **Consume:** `images.processed`
- **Función:**
 - Guardar resultado final (local o S3-like)
 - Opcional: devolver URL

7. API Gateway (Consulta A BD y A S3)

- **Función:**
 - Consulta a la BD si la imagen ya fue procesada
 - Recoge la imagen del S3 si ya fue procesada
- **Tecnología:** FastAPI o Flask

8. minIO (Servicio de Almacenamiento)

- **Función:**
 - Almacenamiento compatible con AWS S3

Topics recomendados

- `images.raw`
- `images.faces_detected`

- `images.age_estimated`
- `images.processed`

Docker Compose

Debe incluir:

- Todos los servicios
- Kafka (con Zookeeper si es necesario)
- Redes internas privadas

Métricas de rendimiento

Cada servicio debe medir:

- Tiempo de procesamiento por evento
- Latencia end-to-end (opcional)
- Throughput (opcional)

3. Dataset

Dataset base: 🖱️ <https://www.kaggle.com/datasets/frabbisw/facial-age>

- Se puede proponer alternativa, pero todos los equipos deben usar la misma

4. Orquestación

El sistema debe ser completamente desacoplado, sin necesidad de un orquestador central. Sin embargo, vamos a implementar un servicio de Orquestador para coordinar el flujo.

El orquestador actúa como el coordinador del pipeline distribuido. Su responsabilidad no es procesar imágenes, sino supervisar el estado de cada trabajo y decidir cuál es el siguiente servicio que debe intervenir. A partir de los eventos emitidos por los distintos microservicios, actualiza el estado de la imagen, resuelve bifurcaciones del flujo, gestiona errores y activa reintentos o rutas alternativas cuando es necesario. De esta forma, se centraliza la lógica de negocio del workflow y se mejora la trazabilidad del procesamiento extremo a extremo

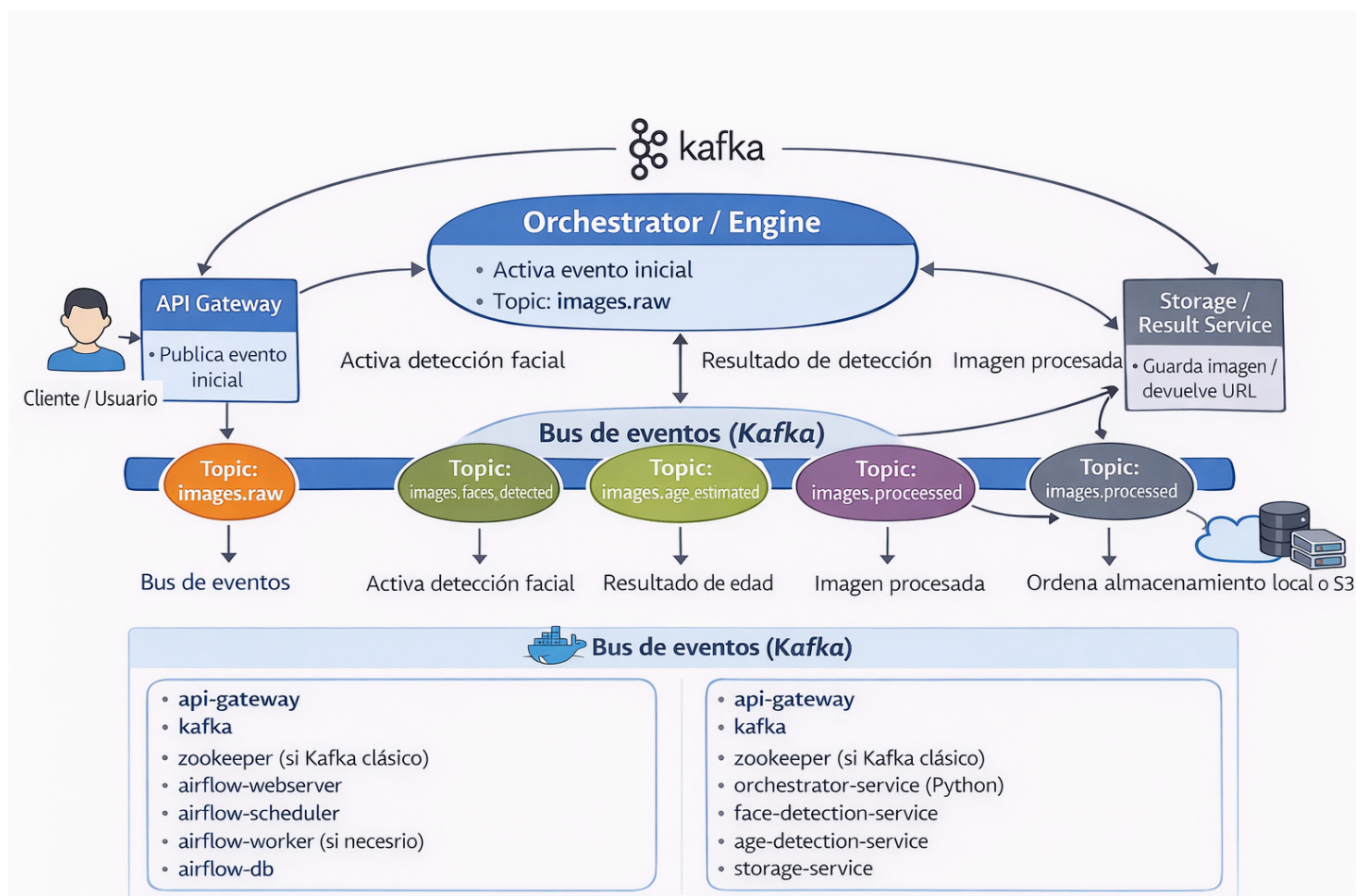
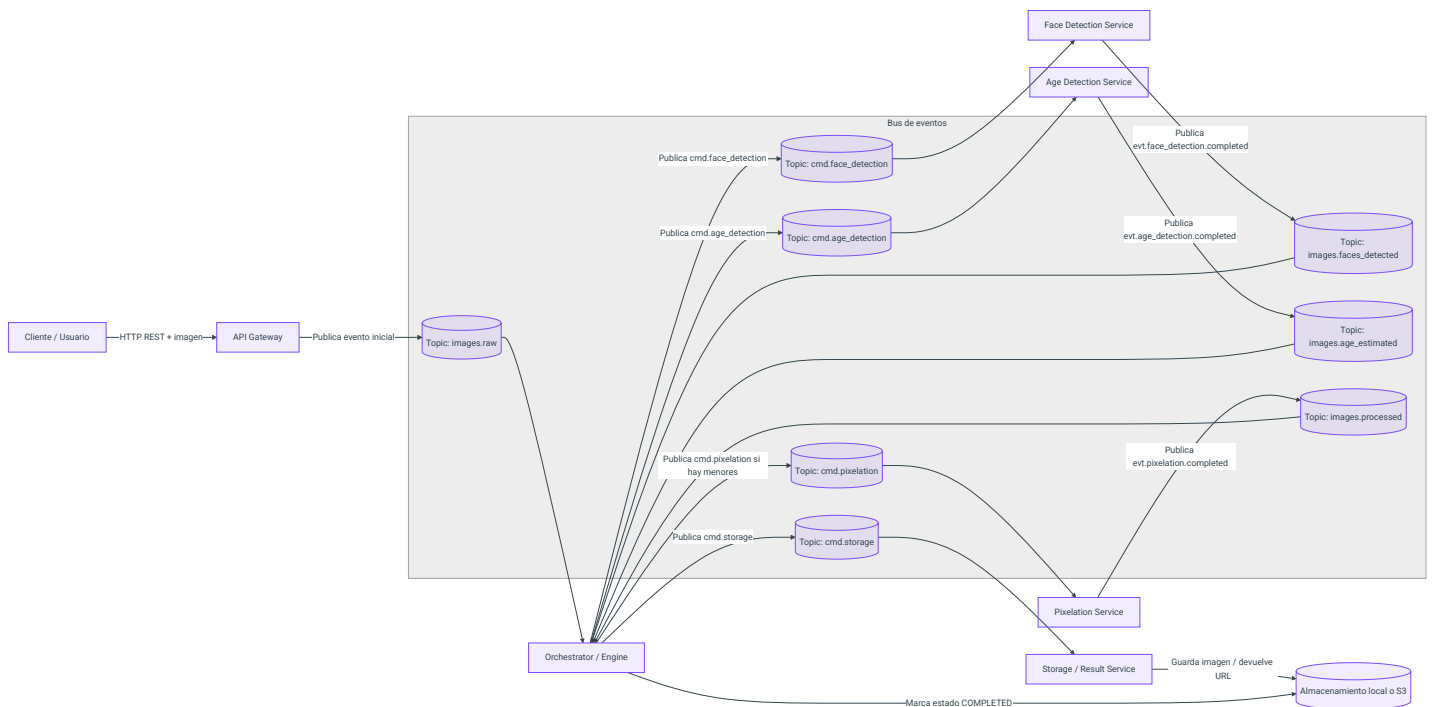
El orquestador sería **el servicio que escucha lo que va pasando en cada etapa y decide cuál es el siguiente paso del procesamiento de la imagen.**

4.1. Flujo Conceptual

1. Cliente -> API Gateway -> `images.raw`
2. Orquestador -> consume `images.raw` -> guarda estado inicial -> publica `cmd.face_detection`
3. Face Detection -> consume `cmd.face_detection` -> publica `evt.face_detection.completed`
4. Orquestador -> decide siguiente paso -> publica `cmd.age_detection` o `cmd.storage`
5. Age Detection -> publica `evt.age_detection.completed`
6. Orquestador -> si hay menores -> `cmd.pixelation` -> si no -> `cmd.storage`

7. Pixelation -> publica evt.pixelation.completed
8. Storage -> publica evt.storage.completed
9. Orquestador -> marca COMPLETED
10. Cliente -> API Gateway -> Recibe metadatos imagen procesada o 404

4.2. Diagrama de flujo



4.3. Tablas

```
CREATE TABLE Solicitud (  
  Id_Solicitud INT PRIMARY KEY,  
  GUID_Solicitud VARCHAR(255),  
  Id_Fichero VARCHAR(255),  
  Inicio_Solicitud DATETIME,  
  Fin_Solicitud DATETIME,  
  Inicio_Deteccion_Caras DATETIME,  
  Fin_Deteccion_Caras DATETIME,  
  Inicio_Almacenamiento_Solicitud DATETIME,  
  Fin_Almacenamiento_Solicitud DATETIME,  
  Num_Imagenes_Total INT,  
  Num_Imagenes_Pixeladas INT,  
  Estado VARCHAR(50)  
);  
  
CREATE TABLE Imagenes (  
  Id_Imagen INT PRIMARY KEY,  
  Id_Solicitud INT,  
  Inicio_Edad DATETIME,  
  Fin_edad DATETIME,  
  Inicio_Pixelado DATETIME,  
  Fin_Pixelado DATETIME,  
  Estado VARCHAR(50),  
  FOREIGN KEY (Id_Solicitud) REFERENCES Solicitud(Id_Solicitud)  
);
```

5. Almacenamiento

Para el almacenamiento de las imagenes utilizaremos el servicio minIO que es totalmente compatible con Amazon S3.

[Enlace](#)

Habr  que crear un contenedor con ese servicio.

6. Entregables

Cada grupo creara un repositorio en GitHub con acceso de lectura para el profesor. Cuyo contenido debe ser:

1. README.md

- C mo ejecutar el sistema (`docker-compose`)
- Estructura del proyecto
- Descripci n de cada servicio
- Topics y flujo de eventos

2. Documentaci n Funcional

- Diagrama de arquitectura
- Flujo de eventos
- Explicaci n del pipeline

3. gesti n de Errores

- Qu  ocurre si falla un servicio o un mensaje no se puede procesar
- Estrategias de manejo (retries, dead-letter queue, etc.)

4. Presentaci n del Proyecto (8 10 diapositivas)

- Diapositivas explicando el proyecto
- Diapositiva con conclusiones
- Diapositiva con propuestas de mejora.

7. Desarrollo del Proyecto

1. Planificación

- División por servicios
- Definición de contratos de eventos (JSON schema)

2. Implementación

- Desarrollo independiente por servicio
- Uso de eventos para integración

3. Pruebas

- Unitarias por servicio
- Integración con Kafka
- Validación del flujo completo

8. Evaluación

1. Funcionalidad

- ¿Se procesan correctamente las imágenes?
- ¿Se detectan y pixelan menores?

2. Arquitectura

- Uso correcto del patrón event-driven
- Desacoplamiento real entre servicios
- Uso adecuado de topics

3. Rendimiento

- Latencia
- Capacidad de procesar múltiples imágenes

4. Calidad del código

- Buenas prácticas
- Estructura clara
- Manejo de errores